Task 1 — Web App QA & Debug Report

App under test: **RealWorld "Conduit" demo** (https://demo.realworld.io) · Tester: Prince Sharma · Role: Automation & QA Developer

> ⚠ **Before you submit — verify each row yourself.** This report lists issues that are commonly reproducible on the RealWorld demo, written in the required template. **Run each repro step on the live app, confirm it still happens, and attach a screenshot.** If any issue no longer reproduces (the demo changes over time), replace it using the bug-hunting checklist at the end — there are more than five candidates there. Only submit bugs you have personally reproduced; you must defend every one in the follow-up call.

Summary

I exercised the main user flows on the RealWorld "Conduit" demo — sign-up, login, create/edit/delete article, comment, favorite, follow, and log out — and probed edge cases (very long input, special characters, empty fields, expired session, rapid actions). Below are the issues found, most severe first, followed by a root-cause analysis of the most impactful one.

Bug Table

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Severity	Suspected Cause
1	**No client-side validation on article creation**	1. Log in. 2. Click "New Article". 3. Leave Title, About, and Body empty. 4. Click "Publish Article".	**Expected:** inline validation blocks submit and shows which fields are required. **Actual:** request is sent; the server returns a 422 and the errors appear only as a terse list (or the form appears to do nothing), which is confusing to users.	High	Front end relies entirely on server-side 422 responses; no pre-submit validation in the React form.
2	**Session/token not cleared cleanly — stale auth state**	1. Log in. 2. Open DevTools → Application → Local Storage, note the `jwt` token. 3. Manually delete or corrupt the token. 4. Navigate / try to post a comment.	**Expected:** app detects the invalid session and redirects to login with a clear message. **Actual:** UI still shows a logged-in state; actions silently fail (401) with no user-facing feedback.	High	Auth state is read from local storage on load but not re-validated; 401 responses aren't globally handled to force logout.
3	**No loading / error state on slow or failed requests**	1. Open DevTools → Network → throttle to "Slow 3G". 2. Load the global feed or open an article.	**Expected:** a spinner/skeleton while loading and a friendly error if the API fails. **Actual:** blank area for several seconds; on failure, nothing renders and no error message is shown.	Medium	Components render only on success; no loading flag or error boundary around API calls.

4	**Long / special-character input breaks layout & is not sanitized in UI**	1. Create an article. 2. In Title, paste a 500-character string with emojis and `<script>`-like text and many `######`. 3. Publish and view it.	**Expected:** title is length-limited or truncated gracefully; markdown/HTML is safely handled. **Actual:** very long titles overflow the card/header layout; unusual input is rendered verbatim, causing visual breakage.	Medium	No max-length on the title field; CSS doesn't truncate; limited input normalization.
5	**Duplicate submit on double-click creates duplicate article / comment**	1. Write a comment or article. 2. Double-click "Publish"/"Post Comment" quickly.	**Expected:** button disables after first click; only one record is created. **Actual:** two identical records can be created because the button isn't disabled during the in-flight request.	Medium	Submit handler doesn't set a "submitting" disabled state while the POST is pending.
6	**Favorite/follow count can desync on rapid toggling**	1. Open an article. 2. Click the Favorite button rapidly several times.	**Expected:** optimistic UI is reconciled with the server; the final count is correct. **Actual:** the count can briefly show a wrong value or get out of sync with the server state until reload.	Low	Optimistic update without request de-duplication / sequencing; race between rapid toggle requests.
7	**Accessibility: form inputs lack associated labels**	1. Open the login or editor page. 2. Inspect inputs / run a Lighthouse or axe accessibility audit.	**Expected:** every input has a programmatic label for screen readers. **Actual:** inputs use placeholders only, with no `<label>` / `aria-label`, failing basic a11y.	Low	Placeholder-as-label anti-pattern in the form components.

Seven candidates are listed so you can drop any that no longer reproduce and still submit five verified issues.

Root-Cause Analysis (Issue #2 — stale authentication state)

What is happening. After login, the app stores a JWT in the browser's local storage and treats "a token exists" as "the user is authenticated." On every page load it reads that token and renders the logged-in UI without checking whether the token is still valid. If the token expires, is deleted, or is corrupted, the interface continues to show the user as logged in.

Why it happens. Authentication state is derived once from local storage at startup and kept in the client store. The app does not re-verify the token against the API (e.g. a `/user` call) on load, and — the core issue — it has no **global handler for 401 Unauthorized** responses. So when the user performs an action (post a comment, publish an article) and the API rejects it with a 401, that failure is swallowed by the individual request instead of triggering a logout. The user sees the action silently do nothing.

> **Impact.** Users believe they are logged in and lose work when actions silently fail; it also looks "broken and flaky," which is exactly the reported symptom. In a real product this erodes trust and generates support tickets.
>
> **How I would fix it.** Two changes. (1) Add a **response interceptor** (e.g. in the Axios/fetch layer) that catches any 401, clears the stored token, resets the auth state, and redirects to the login page with a message like "Your session expired, please log in again." (2) On app load, if a token is present, **validate it** with a lightweight authenticated request (`GET /user`) before rendering the logged-in UI; if it fails, treat the user as logged out. Together these keep the displayed state honest and give clear feedback instead of silent failures.

Bug-Hunting Checklist (extra edge cases to try)

If you need to replace an issue that no longer reproduces, these reliably surface problems on vibe-coded apps:

- **Auth:** wrong password message clarity; sign up with an already-used email; access a protected page while logged out (direct URL).
- **Input:** empty required fields; 1000-char inputs; emojis / RTL text / `<script>` strings; leading-space-only values.
- **Network:** throttle to Slow 3G (loading states); go offline mid-action (error handling); kill the API and watch failure handling.
- **Session:** delete/expire the JWT and keep using the app; log in on two tabs and log out of one.
- **Race/UX:** double-click submit; rapid favorite toggling; browser back button after delete.
- **A11y:** run Lighthouse / axe; try keyboard-only navigation; check color contrast.